

- `main()` → The starting point of the program.
- `runApp(MyApp())` → Launches the Flutter app and loads `MyApp()`.
- b. How `runApp()` Works in Flutter
 - `runApp()` loads the root widget of the app.
 - It sets up the UI hierarchy by displaying the `MyApp()` widget.
 - The entire app structure is built using widgets instead of separate UI files (like XML in Android).
- c. Comparison: `main()` in Java vs. Flutter
 - Java and Flutter both start execution with `main()`, but they handle UI differently.
 - Java (e.g., Android apps) → `main()` only starts logic; UI is built separately using XML.

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

- Flutter/Dart → `main()` immediately builds UI using widgets.
- Key Differences :

Feature	Java (Android Apps)	Flutter (Dart)
UI Handling	UI is built separately using XML	UI is built inside <code>main.dart</code> with widgets
Workflow	<code>main()</code> initializes the app, but UI is handled separately	<code>main()</code> starts the app and UI setup happens immediately
Ease of Management	UI and logic are in separate files	Everything is managed in a single file

Step 3 : Understanding `MyApp` and the Widget Tree

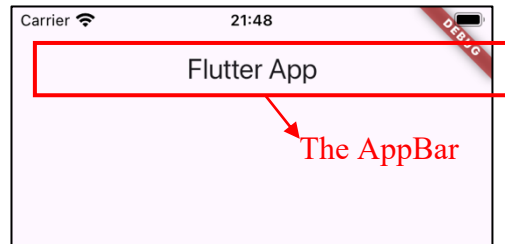
- a. What is `MyApp`?
 - After calling `runApp()`, the app loads the `MyApp` widget.
 - `MyApp` is the root widget of the Flutter application.
 - Every Flutter app is built using widgets, which form the UI.
 - This is an example for basic `MyApp` implementation :

```
class MyApp extends StatelessWidget {
    @override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                appBar: AppBar(
                    title: Text('My Flutter App'),
                ),
                body: Center(
                    child: Text('Hello, world!'),
                ),
            ),
        );
    }
}
```

```

    );
  }
}

```



- Explanation:
 - MyApp → A custom widget that acts as the app's entry point.
 - StatelessWidget → MyApp does not have mutable state.
 - build() method → Defines how the UI is displayed.
 - Returns a widget tree, which is a hierarchical structure of widgets.
- b. How the Widget Tree Works
 - Flutter's UI is structured like a tree (Widget Tree).
 - Each widget can have child widgets, forming a hierarchy.
 - The widget tree defines the structure and relationships between widgets.
 - This is an example of widget tree in MyApp

```

MaterialApp(
  home: Scaffold(
    appBar: AppBar(
      title: Text('My Flutter App'),
    ),
    body: Center(
      child: Text('Hello, world!'),
    ),
  ),
);

```

- Tree structure :

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text ('My Flutter App')
│   └── Body
│       └── Center
│           └── Text ('Hello, world!')

```

- Explanation:
 - MaterialApp → The root widget that wraps everything.
 - Scaffold → Provides the basic screen structure (AppBar, body).
 - AppBar → A bar at the top of the app (for title or actions).

- Text → Displays simple text inside the UI.
- c. Understanding Each Widget in the Tree

Widget	Function	Example
MaterialApp	The root widget that sets up themes & navigation	<code>MaterialApp(home: Scaffold(...))</code>
Scaffold	Provides the basic screen layout (AppBar, body, etc.)	<code>Scaffold(appBar: ..., body: ...)</code>
AppBar	Displays a top navigation bar	<code>AppBar(title: Text('My App'))</code>
Center	Centers its child widget inside the screen	<code>Center(child: Text('Hello'))</code>
Text	Displays simple text	<code>Text('Hello, world!')</code>

- Each widget in Flutter has a specific function and can nest inside other widgets.
- This nesting structure helps organize the UI efficiently.

Step 4 : Understanding the Class and extends

a. What is a Class in Flutter?

- A class is like a blueprint for creating objects (in Flutter, widgets).
- Flutter apps are built using custom widgets, which are defined as classes.
- Each widget class defines how the UI looks and behaves.
- This is how usually we defining a widget class in Flutter :

```
class MyApp extends StatelessWidget {
```

- Explanation:
 - class MyApp → Defines a new widget called MyApp.
 - extends StatelessWidget → MyApp is based on StatelessWidget, meaning it does not change dynamically.

b. What Does extends Mean in Flutter?

- The extends keyword creates a subclass (child class) of another class.
- In Flutter, extends StatelessWidget means the widget is immutable (does not change state).
- A widget only updates when its parent widget rebuilds it.
- This is an example of full Stateless Widget :

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('My Flutter App')),
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```